KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

C PROJECT REPORT ON

THE MINI ATTENDANCE TRACKER

SUBMITTED TO

The Technical Department

KG College of Arts and Science

SUBMITTED BY

Student Name	Roll No.	Course & Batch

Academic Year: **2025 – 2026**

Semester: **I**

DECLARATION

We hereby declare that the project entitled “MINI ATTENDANCE TRACKER” submitted in partial fulfilment of the requirements for the course [**Course Name]** is a **record of our original work** carried out under the guidance of **[Trainer Name]**, Technical Trainer, Technical Department, KGiSL MicroCollege.

We further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

We have acknowledged all sources of information used in this project wherever applicable.

Student Name	Roll No.	Course & Batch	Signature

Place: COIMBATORE

Date: _____

CERTIFICATE

This is to certify that the project entitled **"THE MINI ATTENDANCE TRACKER "** is a **bona fide work** carried out by the following students of **[Class/Batch]**, [**Department Name]**, **KG College of Arts & Science**, in partial fulfilment of the requirements for the course **[Course Name]**.

This project has been completed under my supervision and guidance during the academic period **[Month, Year]**.

It is further certified that this project is the **original work** of the students and has not been previously submitted for the award of any degree, diploma, or certificate at this or any other institution.

Project Team:

S. No.	Name	Roll Number
1.		
2.		
3.		
4.		
5.		

Guide: **Head of the Department**(Technical Trainer, KGM) (Technical Head, KGM)

_____ _____

Place:

Date:

ABSTRACT

The Mini Attendance Tracker is a simple application developed to record and manage student attendance efficiently. In many educational institutions, attendance is often maintained manually, which can be time-consuming and prone to errors. This project aims to provide a basic digital solution for tracking attendance using a computer program.

The system allows the user to enter student details and mark their attendance as present or absent. The attendance information is then stored and can be displayed when needed. This helps in organizing attendance records in a clear and systematic way.

The Mini Attendance Tracker is developed using the C programming language and demonstrates basic programming concepts such as arrays, loops, and file handling. The project is simple, user-friendly, and useful for learning how programming can be applied to solve real-world problems related to data management.

1. INTRODUCTION

1.1 Objective

- To develop a simple system for recording and managing student attendance efficiently.
- To reduce manual errors and save time in maintaining attendance records.
- To store and display attendance details in an organized and easy-to-access format

1.2 Overview

Mini Attendance Tracker is a small software application used to record and manage the attendance of students in a class. It helps teachers or users keep track of who is present or absent on a particular day. Instead of writing attendance manually in registers, this system stores the data digitally.

The project is usually developed using simple programming languages such as C, Java, or Python, and it allows the user to enter student details and mark their attendance. The system can store attendance records and display them whenever needed.

1.3 Features

- **Student Record Management** – Allows adding and storing student names and details.
- **Attendance Marking** – Users can mark students as present or absent easily.
- **Data Storage** – Attendance data can be stored in files (like CSV or text files).
- **View Attendance Records** – Users can view previously saved attendance.
- **Simple User Interface** – Easy to use with basic menu options.
- **Report Generation** – Displays attendance summary for each student.

1. PROGRAM ANALYSIS

2.1 Problem Definition

- 1. Manual Attendance Issues
- Traditional attendance using paper registers is time-consuming and may lead to errors or manipulation.

- 2. Difficulty in Maintaining Records
- Managing and storing attendance data manually becomes difficult when the number of students increases.

- 3. Lack of Quick Access to Attendance Data
- Teachers cannot easily check attendance percentage or individual student records quickly.

- 4. Risk of Data Loss
- Paper-based attendance records can be lost, damaged, or misplaced, making it hard to maintain accurate history.

2.2 Project Overview

Aspect	Description
Language Used	C
Compiler	GCC / Dev-C++ / Turbo C++
Concepts Used	Loops, Functions, Arrays, Conditional Statements
I/O Type	Console-based
Optional	File Handling for saving receipts

2.3 Scopes and Limitations

Scope:

- Helps to **record and manage student attendance** easily in a digital format.
- Allows **quick calculation of attendance percentage** for each student.
- Reduces **manual work and errors** compared to maintaining paper registers.

Limitation

- Suitable only for **small classes or limited number of students**.
- Does not support **advanced features like biometric or online attendance**.
- Requires **basic computer knowledge** to operate the system.

1. SYSTEM DESIGN

3.1 Modules

The Mini Attendance Tracker module is responsible for recording and managing student attendance efficiently. It allows the user to add student details and mark their attendance for specific dates. The system stores the attendance information and helps in viewing or updating records when needed. This module simplifies the manual attendance process and helps maintain accurate attendance data.

Module Name	Description
Student Management Module –	Add , view and manage student details
Attendance marking module	Used to mark student attendance (Present/Absent).
Attendance record module	Used to store and manage attendance data.

View attendance Module	Used to display attendance records of student
Update or delete Module (Optional)	Used to modify or remove attendance details.
Report generation Module	Used to modify the attendance details

Module Interaction Overview

1. Student management module: add,view and manage student details
2. Attendance marking module:mark attendance for student
3. Attendance record module: store and manage attendance data
4. View attendance module: display attendance status for students
5. Update or delete module: modify the student data
6. Report generation mody:generate attendance reports

1. METHODOLOGY

4.1. Problem Analysis

Problem Analysis: Manual attendance tracking is time-consuming and prone to errors. Generating reports is difficult, and data storage is inefficient. There's a need for accuracy, reliability, and a user-friendly interface. Limited accessibility and security concerns are issues. Scalability is a challenge, and automation is needed.

System design: The system has a modular design with student, attendance, and report modules. Database design stores student and attendance data. Input forms handle data entry, and logic marks attendance and generates reports. Error handling mechanisms and a user interface are included. Data validation checks and security measures are part of the design, along with a testing plan.

Program development: Write C code for each module and implement a database using file handling. Develop attendance marking logic and create a report generation function. Add error handling and test individual modules. Integrate modules and test overall functionality. Refine the UI and finalize the code.

Testing and validation : Test student data entry, attendance marking, and report generation. Validate attendance data and check error handling. Test the UI, data accuracy, and scalability. Get user feedback and fix bugs.

Result and documentation: The attendance tracker is functional with accurate records. Reports are generated successfully, and the interface is user-friendly. Error handling works, and the project is documented with an overview, code snippets, and screenshots. Methodology is explained, and a usage guide is provided.

Algorithm : Define attendance marking logic using loops for student iteration. Conditional checks mark present/absent, and attendance counts are updated. Data is stored in files/database and retrieved for reports. Exceptions are handled, and the algorithm is optimized for efficiency.

purpose of the algorithm : The algorithm automates attendance tracking, ensures data accuracy, and reduces manual effort. It enables report generation, improves efficiency, and handles large data. It provides reliability, supports decision-making, and enhances accessibility, simplifying management.

Algorithm in the project: The algorithm includes attendance marking logic, data storage, and retrieval. It handles report generation, error handling, and user input processing. Data validation, conditional checks, and loops are part of it. Calculations like attendance percentage are done, and output is formatted.

*Importance of Algorithm in the Project*by: The algorithm is the core logic for attendance tracking, ensuring accuracy and improving efficiency. It enables report generation, handles large data, and supports scalability. It enhances reliability, simplifies management, reduces errors, and improves user experience

Source code:

```c
#include <stdio.h>

int main() {
    int n, i;
    int attendance[100];
    int present = 0, absent = 0;

    printf("Enter number of students: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        printf("Student %d attendance (1 = Present, 0 = Absent): ", i + 1);
        scanf("%d", &attendance[i]);

        if(attendance[i] == 1) {
            present++;
        } else {
            absent++;
        }
    }

    printf("\n--- Attendance Report ---\n");

    for(i = 0; i < n; i++) {
        if(attendance[i] == 1)
            printf("Student %d : Present\n", i + 1);
        else
            printf("Student %d : Absent\n", i + 1);
    }
```

```
    printf("\nTotal Present: %d", present);

    printf("\nTotal Absent: %d\n", absent);


    return 0;

}
```

–

SOURCE CODE EXPLANATION

1. Header Inclusion

The code starts by including the `stdio.h` header file, which provides input/output functions like `printf` and `scanf`. This is necessary for interacting with the user. The `#include` directive tells the compiler to include the contents of the header file. This is a standard practice in C programming.

2. Main Function

The `main` function is the entry point of the program, where execution begins. It returns an integer value indicating program success or failure. In this case, it returns 0 to indicate successful execution. The `int main()` syntax is used to declare the main function.

3. Variable Declaration

The code declares several integer variables: `n` for the number of students, `i` for loop iteration, `attendance[100]` to store attendance data, and `present` and `absent` counters. These variables are used to store user input and track attendance. The array `attendance[100]` can store data for up to 100 students.

4. User Input

The program prompts the user to enter the number of students using `printf` and stores the input in `n` using `scanf`. It then loops through each student, asking for attendance (1 for present, 0 for absent) and stores it in the `attendance` array. Input validation is not performed in this code.

5. Attendance Processing

Inside the loop, the program checks the attendance value (1 or 0) and increments the `present` or `absent` counter accordingly. This is done using a simple `if-else` statement. The counters keep track of the total number of students present or absent.

6. Report Generation

After collecting attendance data, the program prints an attendance report for each student, indicating whether they were present or absent. It uses a loop to iterate through the `attendance` array and prints the status. The report includes the student number (starting from 1).

7. Total Counts

The program prints the total number of students present and absent at the end of the report. These counts are stored in the `present` and `absent` variables, which are updated during attendance processing. The totals provide a quick summary of the attendance.

8. Loop Constructs

The code uses `for` loops to iterate through students for input and reporting. The loop counter `i` is used to access array elements and display student numbers. Loops are a fundamental construct in C for repetitive tasks.

9. Conditional Statements

The program uses `if-else` statements to check attendance status (1 or 0) and update counters. Another `if-else` is used to print "Present" or "Absent" in the report. Conditional statements control program flow based on conditions.

10. Program Termination

The `main` function returns 0 to indicate successful program execution. This is a standard practice in C programming. The program terminates after executing all statements in the `main` function. No error handling is implemented in this code.

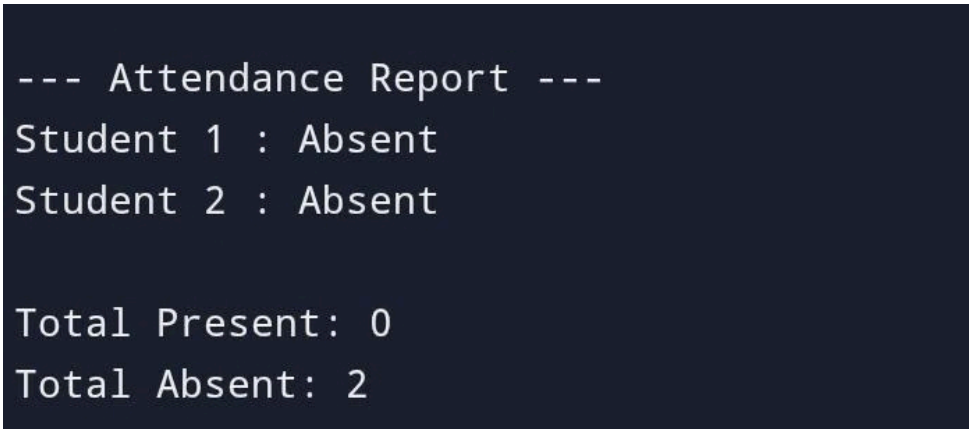
Want to *add features* or *improve* the code? 😊

1. INPUT AND OUTPUT SCREENS

Input Screen :

```
Enter number of students: 2
Student 1 attendance (1 = Present, 0 =
    Absent): 3
Student 2 attendance (1 = Present, 0 =
    Absent): 2
```

Output Screen:



1. DISCUSSION & CONCLUSION

7.1 Discussion

This project successfully automates restaurant billing by:

Using structured programming and modular design.

Applying loops, arrays, and structures for data management.

Demonstrating real-world utility with file saving (optional).

The system is efficient, user-friendly, and suitable for small restaurants.

7.2 conclusion

The project meets its intended objectives:

Simple yet functional billing automation.

Demonstrated real-life application of C programming concepts.

Easy to extend for further features like login system, discount, and database storage.

1. **REFERENCES**

- geeksforgeeks
- tensorflow
- opencvdovumentation
- research gate